



Опрос про ИИ внутри разработки

[Войти](#)

Geronom

4 минуты назад

Бенчмаркая foreach и эnumераторы: аллокации, которые прятались 10 лет

 Простой 17 мин 5

.NET*, C#*, Программирование*

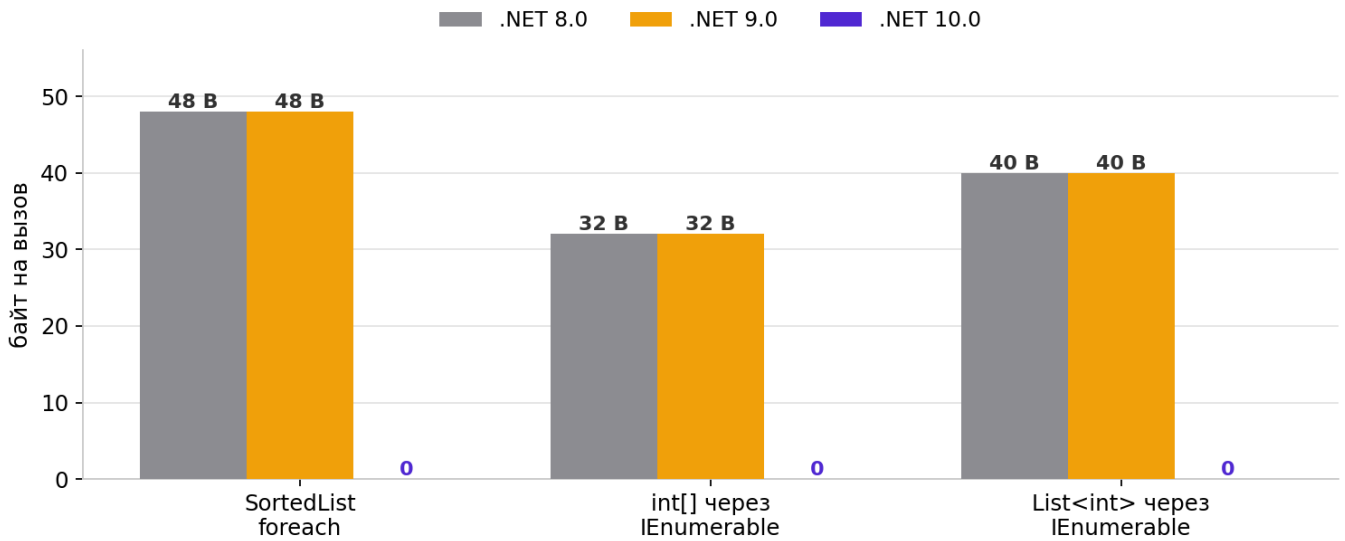
Уважаемые читатели, в этой статье я хочу рассказать о скрытых аллокациях в обычном foreach и представить свои выводы. Тесты сделаны на .net 8, .net 9 и .net 10 одновременно, чтобы было видно, как одна и та же строчка кода ведет себя на разных рантаймах.

Все тесты сделаны с использованием BenchmarkDotNet, полный код представлен на гите, так что каждый может проверить результаты и сделать свои выводы.

Будет три истории, и все три про foreach:

- SortedList из стандартной библиотеки аллоцирует память на каждый foreach, а Dictionary — нет. Из-за одной строчки в сигнатуре метода. И починить это по-нормальному нельзя.
- .NET 10 научился убирать такие аллокации вообще без изменения вашего кода. И по пути выяснилось, что одна старая оптимизация мешала новой.
- У новой оптимизации есть задокументированная самим Microsoft граница — на больших коллекциях она может срываться. Я пошел ловить этот обрыв с бенчмарком — и получилось интереснее, чем я ожидал.

Аллокации на один foreach: было и стало



Одинаково на всех 4 машинах. .NET 10: 10.0.1 / 10.0.5 / 10.0.9

Рис. 1. Аллокации на один foreach: .NET 8/9 против .NET 10 (одинаково на всех четырех машинах)

Каждое утверждение подкреплено ссылкой на GitHub dotnet/runtime или на официальный блог .NET — специально собрал пруфы, чтобы можно было пойти и проверить первоисточник, а не верить мне на слово. Все ссылки продублированы списком в конце.

Тесты гонялись на четырех машинах — от игровых десктопов до двухпроцессорных серверов:

Машина	CPU	ОС	.NET 10 / 9 / 8
№1	AMD Ryzen 9 5950X, 16 ядер	Windows 10	10.0.5 / 9.0.15 / 8.0.14
№2	Intel Core i9-10900KF, 10 ядер	Windows 10	10.0.9 / 9.0.17 / 8.0.28
№3	2 x Intel Xeon Silver 4314, 32 ядра	Windows Server 2022	10.0.1 / 9.0.5 / 8.0.16
№4	2 x Intel Xeon Silver 4314, 32 ядра	Windows Server 2022	10.0.1 / 9.0.5 / 8.0.16

BenchmarkDotNet везде v0.15.8. Подробные таблицы по ходу статьи — с машины №2, по остальным привожу сводные сравнения. Обратите внимание на версии .NET 10: 10.0.1, 10.0.5 и 10.0.9 — три разные сервисные сборки, это еще сыграет роль в третьей истории.

Немного теории

Про foreach написано немало, поэтому сильно углубляться не буду, но немного напишу, что да как — иначе дальше будет непонятно, откуда берутся аллокации там, где нет ни одного new.

Энумератор — это маленький объект-курсор, который умеет две вещи: сдвинуться на следующий элемент (MoveNext()) и отдать текущий (Current). Когда вы пишете foreach, компилятор за кулисами берет у коллекции энумератор и крутит его в цикле while. То есть каждый foreach — это создание одного энумератора.

Дальше главный вопрос: где живет этот энумератор — на стеке (бесплатно, сборщик мусора про него даже не узнает) или на куче (аллокация, которую потом придется собирать GC)?

Тут есть неочевидный факт: foreach работает не через интерфейс, а через утиную типизацию — по классике: если это выглядит как утка, плавает как утка и крикает как утка — значит, это утка. Компилятору не нужен IEnumerable — ему нужен просто метод с именем GetEnumerator(). И компилятор берет тот метод, который видит по типу переменной. Поэтому у List<T> внутри целых три метода GetEnumerator:

```
// Так выглядит List<T> внутри (упрощенно):

// 1. Публичный. Возвращает struct КАК ЕСТЬ. Его берет foreach,
//    когда тип переменной - List<T>. Ноль аллокаций.
public List<T>.Enumerator GetEnumerator()
    => new List<T>.Enumerator(this);

// 2. Явная реализация интерфейса. Возвращает ТОТ ЖЕ struct,
//    но уже как интерфейс. Struct в интерфейсной переменной жить
//    не может - происходит боксинг в объект на куче.
IEnumerator<T> IEnumerable<T>.GetEnumerator()
    => new List<T>.Enumerator(this);

// 3. То же самое для старого необобщенного IEnumerable.
IEnumerator IEnumerable.GetEnumerator()
    => new List<T>.Enumerator(this);
```

Объяснить с  SourceCraft

Боксинг — это когда struct нужно передать туда, где ждут ссылку на объект. Рантайм выделяет объект на куче и копирует туда значение структуры. Каждый боксинг — это аллокация, и ее видно в колонке Allocated у BenchmarkDotNet.

А теперь пример, вокруг которого крутится вся статья. Код одинаковый, разница только в типе переменной:

```
var list = new List<int> { 1, 2, 3 };

// Тип переменной - List<int>. foreach берет метод №1.
// Struct-эnumератор живет на стеке. Аллокаций: 0.
foreach (var n in list) { }

// Тип переменной - IEnumerable<int>. foreach берет метод №2.
// Тот же struct, но боксированный на кучу.
// Аллокаций: 1 объект на каждый foreach.
IEnumerable<int> seq = list;
foreach (var n in seq) { }
```

Объяснить с 

Вот и вся теория. Никакого new в коде нет, а аллокация есть. Дальше — три истории, где эта мелочь оборачивается интересными последствиями, и бенчмарки, чтобы все пощупать руками.

История первая: SortedList, который аллоцирует, и никто не может это починить

В январе 2023 года разработчик под ником emilsteen завел в dotnet/runtime issue №81128. История максимально жизненная: у человека на проде foreach по маленькому SortedList на 15 элементов, он гоняет его миллион раз и видит в BenchmarkDotNet аллокации. А точно такой же foreach по Dictionary — ноль байт. Он не поленился и раскопал причину сам.

Причина — одна строчка. Сравните сигнатуры публичных методов GetEnumerator у двух коллекций:

```
// Dictionary<TKey, TValue> - возвращает конкретный struct:
public Dictionary<TKey, TValue>.Enumerator GetEnumerator()

// SortedList<TKey, TValue> - возвращает интерфейс:
public IEnumerator<KeyValuePair<TKey, TValue>> GetEnumerator()
```

Объяснить с 

Внутри SortedList эnumератор тоже структура (когда-то, еще после .NET Framework, его переделали из класса в структуру именно ради экономии). Но публичный метод отдает его через интерфейс — и структура боксится прямо на выходе. foreach по типу

переменной SortedList находит именно этот метод. Итог: каждый foreach по SortedList — гарантированный боксинг, и никакая структура внутри не спасает.

Автор issue проверил и это: если поменять возвращаемый тип на конкретный Enumerator — аллокация уходит, а сам GetEnumerator по его замерам становится минимум вдвое быстрее. Он же нашел ту же болячку в ReadOnlyDictionary и еще нескольких коллекциях.

Кстати, на Хабре этот факт уже всплывал — но именно как загадка. В комментариях к статье про Dictionary и SortedDictionary (habr.com/ru/articles/784852/comments) юзер vvdev, гоняя свои бенчмарки, заметил: SortedList выделяет память в куче при каждом вызове GetEnumerator, константные 48 байтов — и честно добавил, что не смотрел, что именно там выделяется.

Вот эти 48 байтов — это и есть боксированный энумератор: заголовок объекта плюс поля структуры (точный размер зависит от типов ключа и значения). То есть на эти 48 байтов уже натыкались, но что это и откуда — так и осталось без ответа. Сейчас разберемся.

Почему Microsoft просто не поменяет одну строчку?

Вот тут самое интересное. Отвечал emilsteen в тредке сам Стивен Тауб из команды .NET. Почему энумератор аллоцируется, он объяснил одним словом: Boxing. А на предложение поменять возвращаемый тип ответил: это ломающее изменение (breaking change) — возвращаемый тип входит в сигнатуру метода, и весь уже скомпилированный чужой код, который делает foreach по SortedList, сломается.

Issue закрыли как answered: по API все работает как задумано, просто задумано было неудачно, и теперь это навсегда. Ровно с той же дилеммой, кстати, столкнулись в Google Protobuf (github.com/protocolbuffers/protobuf/issues/13503) — у них коллекции боксят энумераторы по той же причине, и там тоже пришли к выводу: поменять сигнатуру нельзя, можно только добавить рядом новый метод с другим именем.

Мораль первой истории простая: вся разница между «ноль аллокаций» и «аллокация на каждый foreach» — это возвращаемый тип у GetEnumerator. Ошиблись в нем даже авторы стандартной библиотеки, а исправить уже нельзя — сигнатуру публичного метода не поменяешь. Теперь проверим цифрами.

► [Скрытый текст](#)

Результаты:

Method	Runtime	Mean	Ratio	Allocated
Dictionary_Foreach	.NET 10.0	10.16 ns	1.00	-
SortedList_Foreach	.NET 10.0	12.18 ns	1.20	-
Dictionary_AsEnumerable_Foreach	.NET 10.0	12.53 ns	1.23	-
Dictionary_Foreach	.NET 8.0	11.71 ns	1.15	-
SortedList_Foreach	.NET 8.0	37.14 ns	3.66	48 B
Dictionary_AsEnumerable_Foreach	.NET 8.0	35.10 ns	3.46	48 B
Dictionary_Foreach	.NET 9.0	10.33 ns	1.02	-
SortedList_Foreach	.NET 9.0	37.74 ns	3.72	48 B
Dictionary_AsEnumerable_Foreach	.NET 9.0	84.98 ns	8.37	48 B

Рис. 2. SortedListVsDictionary: результаты на машине №2

Смотрим на .net 8 и .net 9: SortedList_Foreach — 48 байтов на каждый foreach, Dictionary_Foreach — прочерк. И обратите внимание: это ровно те же 48 байтов из хабр-комментария, о котором я писал выше — теперь мы знаем, что это за байты: боксированный struct-эnumератор (заголовок объекта плюс поля структуры). Контрольный Dictionary_AsEnumerable_Foreach тоже показал 48 B — то есть мы руками превратили Dictionary в SortedList одним приведением типа, как и обещала теория.

А теперь .net 10 — и интрига разрешилась: все три строчки по нулям. Escape analysis дотянулся и до SortedList: JIT заинлайнил GetEnumerator, увидел, что боксированный эnumератор не покидает метод, и разместил его на стеке. Причем не только память — время: 12.18 ns против 37.14 ns на .net 8, в три раза быстрее без единой измененной строчки кода.

Получается интересная штука: в исходниках SortedList ничего не поменялось — GetEnumerator как возвращал интерфейс, так и возвращает. Проблему, которую нельзя было исправить в коде библиотеки, исправил уровень ниже — рантайм.

Отмечу тот факт, что смотреть тут надо в первую очередь на колонку Allocated, а не на время: вся история — про память и давление на GC, и ноль против ненуля — это диагноз. А теперь проверим, что это не особенность одной машины. SortedList_Foreach на всех четырех:

Runtime	№1 Ryzen 5950X	№2 i9-10900KF	№3 Xeon (сервер)	№4 Xeon (сервер)
.NET 8.0	38.9 ns / 48 B	37.1 ns / 48 B	62.2 ns / 48 B	65.9 ns / 48 B
.NET 9.0	33.2 ns / 48 B	37.7 ns / 48 B	61.5 ns / 48 B	68.7 ns / 48 B
.NET 10.0	10.2 ns / 0	12.2 ns / 0	20.9 ns / 0	22.3 ns / 0

Рис. 3. SortedList_Foreach на четырех машинах: время / Allocated

Картина одинаковая везде: 48 байтов на .net 8/9 и ноль на .net 10 — на трех разных сервисных сборках десятки (10.0.1, 10.0.5, 10.0.9) и на разном железе.

Тут я удивился, но не десятке, а девятке. Посмотрите на Dictionary_AslEnumerable_Foreach: на .net 9 он в 2.4 раза медленнее, чем тот же код на .net 8. Сначала я списал это на кривой прогон, но нет — эффект воспроизвелся на всех четырех машинах и на трех разных версиях .NET 9 (9.0.5, 9.0.15, 9.0.17):

Runtime	№1 Ryzen	№2 i9	№3 Xeon	№4 Xeon
.NET 8.0	35.2 ns	35.1 ns	61.3 ns	59.8 ns
.NET 9.0	91.1 ns	85.0 ns	144.6 ns	148.7 ns
.NET 10.0	11.7 ns	12.5 ns	21.7 ns	24.2 ns

Рис. 4. Dictionary_AslEnumerable_Foreach на четырех машинах

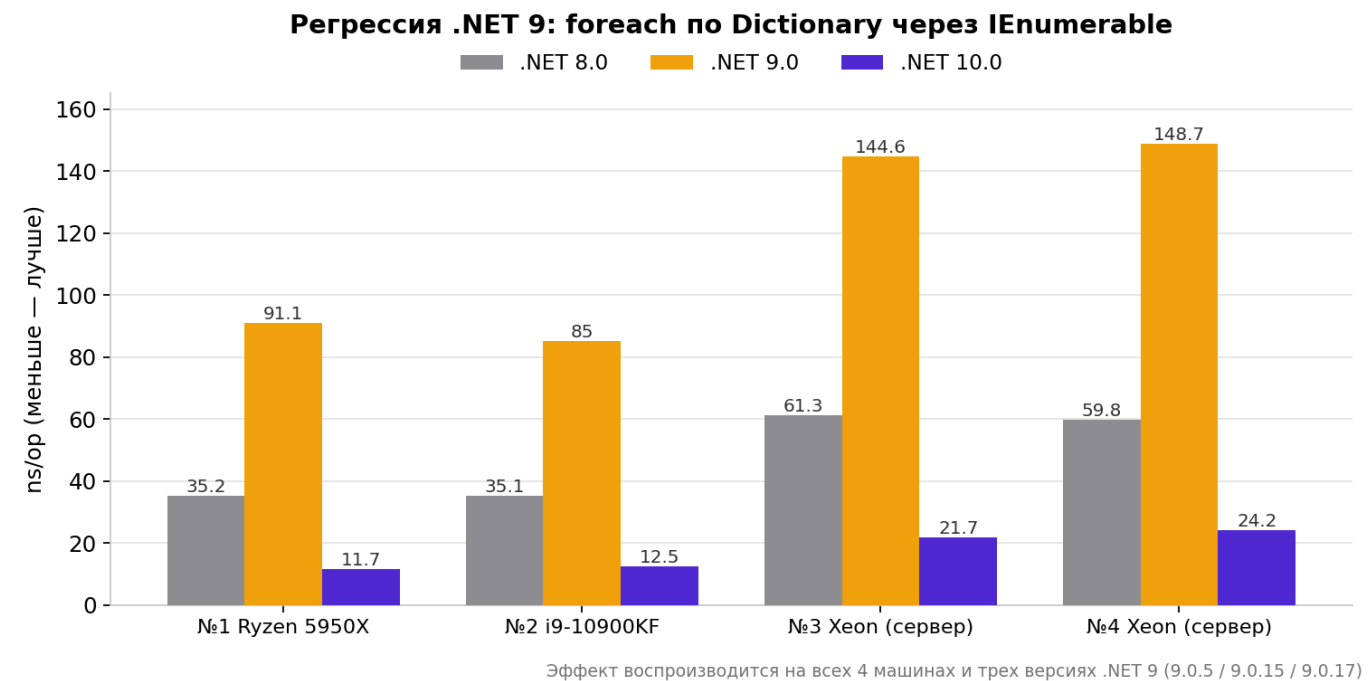


Рис. 5. Регрессия .NET 9 на переборе Dictionary через интерфейс

То есть в .net 9 живет стабильная регрессия на переборе словаря через интерфейс, которую в .net 10 не просто починили, а перепрыгнули с запасом. Причину я не раскапывал — похоже на неудачную эвристику девиртуализации именно в девятке. Если кто-то в комментариях докопается до конкретного изменения — будет интересно.

История вторая: .NET 10 заходит с другой стороны

Итак, чинить сигнатуры в API нельзя. Но в .NET 10 команда рантайма пошла другим путем: раз нельзя убрать боксинг из кода, пусть его убирает JIT-компилятор на лету. Это называется деабстракция (de-abstraction), и это официально одно из главных направлений релиза.

Здесь работают два механизма:

- Escape analysis (анализ убегания). JIT смотрит на объект и спрашивает: этот объект покидает метод? Сохраняется в поле, возвращается наружу, уходит в неизвестный чужой код? Если нет — значит, время его жизни ограничено методом, и объект можно не выделять на куче, а разместить прямо на стеке. Стек освобождается сам при выходе из метода, GC вообще не в курсе, что объект существовал.
- PGO и guarded devirtualization. Рантайм какое-то время наблюдает за методом и собирает профиль: какой конкретный тип реально приходит в переменную IEnumerable. Если в 99% случаев это int[] — JIT генерирует отдельную быструю ветку кода именно под int[], где все вызовы эnumератора уже не виртуальные, инлайнятся, и эnumератор можно стек-аллоцировать. На случай, если придет что-то другое, остается запасная медленная ветка (потому и guarded — с охраной).

Пруфы. Официальный план деабстракции — [issue №108913](#), там прямым текстом: благодаря PR №108153 мы можем девиртуализовать и заинлайнить конструктор эnumератора и вызовы на нем, а в ряде случаев даже стек-аллоцировать эnumератор — и в таблице бенчмарков .NET 10 аллокаций больше нет.

То же самое — в официальной документации [What's new in .NET 10 runtime](#) и в огромном посте Стивена Тауба [Performance Improvements in .NET 10](#). Тауб отдельно пишет: с прогрессом .NET 10 это происходит очень часто для массивов и List<T>, а интерфейс IEnumerable<T> даже поместили как [Intrinsic], чтобы JIT мог рассуждать о нем напрямую.

Как одна оптимизация мешала другой: пустой массив

А теперь самое интересное из [issue №108913](#) — то, ради чего отчасти и писалась эта статья. Когда команда JIT взялась стек-аллоцировать эnumераторы массивов, она уперлась в... собственную старую оптимизацию.

Дело вот в чем. Эnumератор массива — это класс SZGenericArrayEnumerator<T>. И в его создании давным-давно сделали экономию: для пустого массива новый эnumератор не создается, а возвращается один общий статический экземпляр (синглтон). Логика железная — зачем плодить объекты ради перебора нуля элементов.

Но для escape analysis это подстава: теперь в точке, где крутится цикл, JIT не может сказать, какой именно объект он итерирует — свежесозданный (его можно на стек) или

тот самый общий синглтон (его на стек нельзя, он общий на весь процесс). Неоднозначность — и анализ пасует. В issue это описано прямым текстом: из-за оптимизации для пустых массивов в точках использования эnumератора возникает неопределенность, какой объект итерирует.

Анти-аллокационный трюк десятилетней давности блокировал анти-аллокационную оптимизацию 2025 года!

Пришлось учить JIT разруливать оба варианта (это и называется conditional escape analysis — стек-аллокация с проверкой условия на входе). По-моему, это лучшая иллюстрация того, почему в перформансе нельзя ничего принимать на веру: даже оптимизации внутри самого рантайма могут конфликтовать друг с другом.

► [Скрытый текст](#)

Результаты:

Method	Runtime	Mean	Ratio	Allocated
Array_Direct	.NET 10.0	131.1743 ns	1.000	-
Array_AsEnumerable	.NET 10.0	152.9469 ns	1.166	-
Array_ViaHelper	.NET 10.0	150.8422 ns	1.150	-
EmptyArray_AsEnumerable	.NET 10.0	0.0003 ns	0.000	-
Array_Direct	.NET 8.0	139.8498 ns	1.066	-
Array_AsEnumerable	.NET 8.0	582.4156 ns	4.440	32 B
Array_ViaHelper	.NET 8.0	580.1560 ns	4.423	32 B
EmptyArray_AsEnumerable	.NET 8.0	2.7780 ns	0.021	-
Array_Direct	.NET 9.0	128.8994 ns	0.983	-
Array_AsEnumerable	.NET 9.0	682.5440 ns	5.203	32 B
Array_ViaHelper	.NET 9.0	679.2527 ns	5.178	32 B
EmptyArray_AsEnumerable	.NET 9.0	2.2612 ns	0.017	-

Рис. 6. ArrayDeabstraction: результаты на машине №2

Тут все сбылось буквально. На .net 8 и .net 9 массив через интерфейс — это 32 байта боксинга на каждый вызов и просадка в 4.4–5.2 раза по времени. На .net 10 — прочерк в Allocated, а время 152.9 ns: интерфейсный цикл стал всего на 17% медленнее прямого — при том же самом коде.

Это и есть деабстракция в действии, и это тот самый ответ, почему нельзя верить статьям с выводами уровня «foreach по IEnumerable всегда боксит»: на .net 8 — да, на .net 10 — уже нет. Сводка по всем машинам (Array_AslEnumerable):

Runtime	№1 Ryzen	№2 i9	№3 Xeon	№4 Xeon
.NET 8.0	372.7 ns / 32 B	582.4 ns / 32 B	1.20 us / 32 B	1.25 us / 32 B
.NET 9.0	439.4 ns / 32 B	682.5 ns / 32 B	1.30 us / 32 B	1.32 us / 32 B
.NET 10.0	116.2 ns / 0	152.9 ns / 0	287.9 ns / 0	299.5 ns / 0

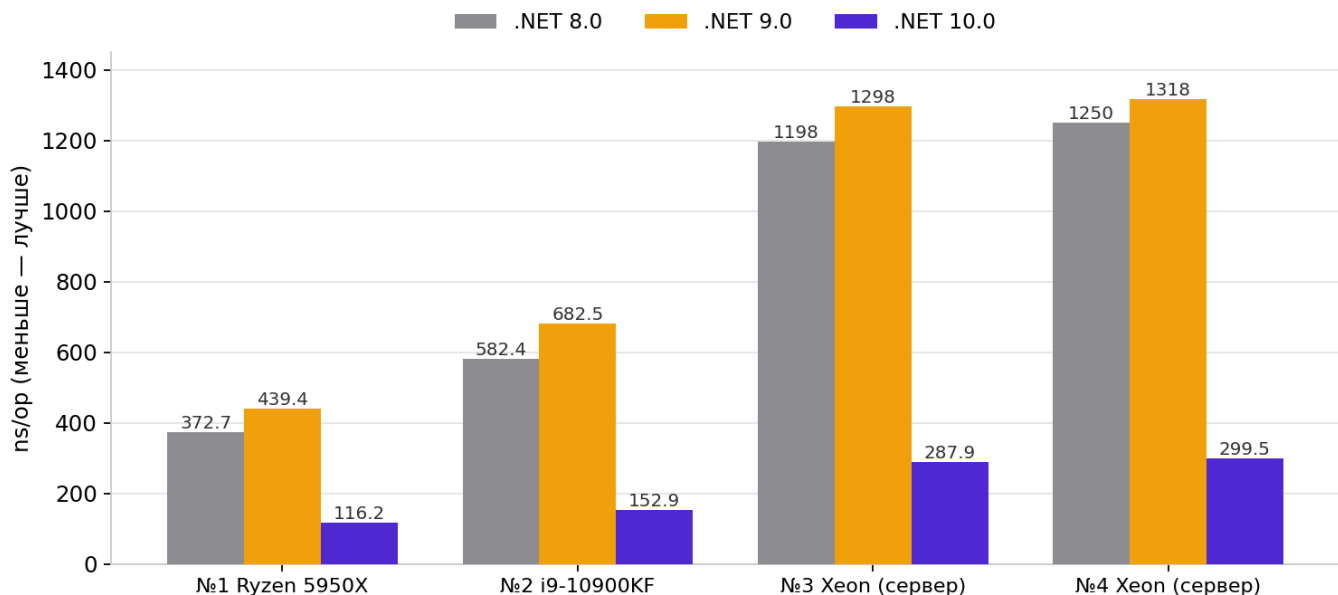
Рис. 7. Array_AslEnumerable на четырех машинах: время / Allocated

Отдельно повеселила машина №1: на Ryzen интерфейсный вариант на .net 10 (116.2 ns) обогнал даже прямой foreach по массиву (124.9 ns). Нет, интерфейсы не стали быстрее массивов — после всех оптимизаций JIT генерирует для обоих вариантов практически одинаковый цикл, и кто из двух окажется на пару процентов быстрее, решают уже случайные мелочи вроде того, как код лег в памяти. Но сам факт показателен: раньше интерфейс проигрывал в четыре раза, теперь эти два варианта почти неотличимы.

Попутно два наблюдения по таблице:

- .net 9 в этом тесте медленнее .net 8 (682 против 582 ns) — и это уже второй случай, где девятка проседает относительно восьмерки. Прогресс между версиями не обязан быть монотонным: эвристики JIT перекраиваются, и отдельные сценарии могут временно деградировать.
- EmptyArray_AslEnumerable на .net 10 — 0.0003 ns. Это не опечатка и не сломанный замер: JIT понял, что перебор пустого массива не делает ничего, и выкинул весь цикл целиком — метод схлопнулся до «верни ноль». Синглтон-эnumератор, который мешал escape analysis, в итоге сам был съеден оптимизатором вместе с циклом.

foreach по массиву через IEnumerable<int>: время на вызов



Allocated: .NET 8/9 — 32 В на вызов, .NET 10 — 0 В. Данные: BenchmarkDotNet v0.15.8

Рис. 8. foreach по массиву через IEnumerable: время на вызов

Смотрим машинный код

Колонке Allocated можно верить, но раз уж мы охотимся на то, чего нет в исходниках, финальный пруф — в машинном коде. JIT умеет показывать сгенерированный код через переменную окружения, без каких-либо инструментов. Чтобы вывод был чистым, я вынес тот же foreach по IEnumerable<int> в отдельную мини-программу (метод SumOverInterface, код в репозитории) и снял листинги на машине №2:

```
dotnet build -c Release
set DOTNET_JitDisasm=SumOverInterface*
bin\Release\net8.0\DisasmProof.exe > disasm_net8.txt 2>&1
bin\Release\net10.0\DisasmProof.exe > disasm_net10.txt 2>&1
```

Объяснить с

JIT печатает метод несколько раз — по мере прогрева он перекомпилирует его от простого tier 0 до оптимизированного tier 1. Нас интересуют финальные tier 1-версии. Сначала .net 8 (листинги сокращены до смысловых строк):

```
; .NET 8, Tier1, optimized using Dynamic PGO ; всего 460 байт кода

call [r11]IEnumerable`1[int]:GetEnumerator()
mov gword ptr [rbp-0x30], rcx ; ссылка на объект-эnumератор В КУЧ

G_M000_IG06: ; тело цикла, каждая итерация:
cmp rsi, rdi ; тип эnumератора все еще тот, что
```

```

    jne     SHORT G_M000_IG10          ; нет - уходим на виртуальные вызовы
    ...
G_M000_IG10:
    call    [r11]IEnumerator:MoveNext() ; виртуальный вызов (медленный путь)
    ...
G_M000_IG24:
    call    [r11]IDisposable:Dispose() ; виртуальный Dispose в конце

```

Объяснить с 

Что тут происходит. Восьмерка с включенным PGO уже не так проста: она угадала конкретный тип эnumератора и заинлайнила его MoveNext и Current (это guarded devirtualization). Но два момента никуда не делись: сам объект эnumератора создается в куче внутри неvirtуализованного вызова GetEnumerator — это и есть те 32 байта из таблиц (gword в листинге — ссылка на объект в куче, которую отслеживает GC), и проверка типа повторяется на каждой итерации цикла.

Теперь .net 10:

```

; .NET 10, Tier1                                     ; всего 193 байта кода

    mov     rax, 0x7FF9F36CB188                      ; method table типа int[]
    cmp     qword ptr [rcx], rax                     ; проверка типа ОДИН раз, на входе
    jne     SHORT G_M000_IG07                         ; не массив - запасной общий путь
    mov     eax, dword ptr [rcx+0x08]                 ; длина массива

G_M000_IG03:                                         ; весь foreach целиком:
    add     ebx, dword ptr [rcx+4*r8+0x10]           ; читаем элемент прямо из массива
    inc     edx
    cmp     edx, eax
    jb     SHORT G_M000_IG03

```

Объяснить с 

Вот ради этих строчек все и затевалось. GetEnumerator на быстром пути не вызывается вообще. Эnumератора нет — ни в куче, ни на стеке: JIT его полностью выкинул и переписал foreach в индексный проход по массиву, тот самый, который for.

Проверка типа выполняется один раз на входе в метод, а не на каждой итерации. Размер кода — 193 байта против 460. Аллокация исчезла не потому, что ее куда-то спрятали: ее физически нет в машинном коде.

История третья: оптимизация, которая исчезает с размером списка

Если бы на этом все закончилось, вывод был бы простой: обновляйтесь на .NET 10 и забудьте про боксинг эnumераторов. Но нет. У этой оптимизации есть граница, и она задокументирована самим же Таубом в том же посте про .NET 10. Причем проходит она в неожиданном месте — по размеру коллекции.

Тауб описывает два эффекта:

- Редкая ветка. Внутри MoveNext у List<T> есть запасной метод MoveNextRare — он вызывается один раз в самом конце перебора, чтобы завершить итерацию. То есть на список из 10 элементов приходится 10 обычных шагов и один вызов MoveNextRare, а на список из миллиона — миллион шагов и все равно один MoveNextRare.

Дальше вступает профилировка. JIT решает, какие методы заинлайнить, глядя на статистику вызовов: часто вызывается — инлайним, почти не вызывается — не тратим на него место. На коротком списке MoveNextRare срабатывает каждый десятый вызов — для JIT это нормальная рабочая ветка, он ее инлайнит, весь эnumератор целиком разбирается по косточкам и ложится на стек.

На длинном списке тот же MoveNextRare — один вызов на миллион, по статистике им можно пренебречь, и JIT его не инлайнит. А вот теперь ловушка: раз метод не заинлайнен, эnumератор нужно передать в него как обычный объект — то есть отдать наружу. Для escape analysis «отдать наружу» значит «объект убегает из метода», и класть его на стек больше нельзя. Стек-аллокация срывается — из-за метода, который вызовется один раз.

- OSR против PGO. Если метод крутит очень длинный цикл, рантайм не ждет его окончания, а перекомпилирует метод прямо посреди выполнения — это называется OSR (On-Stack Replacement). Проблема: OSR-версии методов не собирают PGO-инструментацию. А без профиля для хвоста метода — того места, где вызывается Dispose эnumератора — JIT не может сделать guarded devirtualization, и вся цепочка оптимизаций рассыпается.

Итог по описанию должен быть парадоксальный: один и тот же foreach по IEnumerable<int> на .NET 10 дает 0 B на маленьком списке и снова начинает аллоцировать на большом. Звучит как отличный материал для бенчмарка — берем Params на три размера и идем ловить обрыв.

► [Скрытый текст](#)

Результаты (для компактности сгруппировал по размеру):

Method	Runtime	Size	Mean	Ratio	Allocated
List_Direct	.NET 10.0	10	5.638 ns	1.00	-
List_AsEnumerable	.NET 10.0	10	7.935 ns	1.41	-
List_ViaHelper	.NET 10.0	10	7.963 ns	1.41	-
List_Direct	.NET 8.0	10	5.651 ns	1.00	-
List_AsEnumerable	.NET 8.0	10	21.335 ns	3.78	40 B
List_ViaHelper	.NET 8.0	10	21.681 ns	3.85	40 B
List_Direct	.NET 9.0	10	5.679 ns	1.01	-
List_AsEnumerable	.NET 9.0	10	20.835 ns	3.70	40 B
List_ViaHelper	.NET 9.0	10	20.754 ns	3.68	40 B
List_Direct	.NET 10.0	1000	407.838 ns	1.00	-
List_AsEnumerable	.NET 10.0	1000	417.770 ns	1.02	-
List_ViaHelper	.NET 10.0	1000	418.381 ns	1.03	-
List_Direct	.NET 8.0	1000	540.332 ns	1.32	-
List_AsEnumerable	.NET 8.0	1000	1,354.263 ns	3.32	40 B
List_ViaHelper	.NET 8.0	1000	1,347.821 ns	3.30	40 B
List_Direct	.NET 9.0	1000	406.826 ns	1.00	-
List_AsEnumerable	.NET 9.0	1000	1,362.683 ns	3.34	40 B
List_ViaHelper	.NET 9.0	1000	1,361.274 ns	3.34	40 B
List_Direct	.NET 10.0	1000000	416,097.380 ns	1.00	-
List_AsEnumerable	.NET 10.0	1000000	419,686.637 ns	1.01	-
List_ViaHelper	.NET 10.0	1000000	417,819.775 ns	1.00	-
List_Direct	.NET 8.0	1000000	471,606.281 ns	1.13	-
List_AsEnumerable	.NET 8.0	1000000	1,334,484.029 ns	3.21	40 B
List_ViaHelper	.NET 8.0	1000000	1,340,837.709 ns	3.22	40 B
List_Direct	.NET 9.0	1000000	415,001.416 ns	1.00	-
List_AsEnumerable	.NET 9.0	1000000	1,389,870.833 ns	3.34	40 B
List_ViaHelper	.NET 9.0	1000000	1,364,448.919 ns	3.28	40 B

Рис. 9. ListSizeCliff: результаты на машине №2, три размера списка

Сначала подтвержденная часть. На .net 8 и .net 9 картина железная: 40 байтов на боксинг эnumератора на любом размере и трехкратная просадка по времени. На .net 10 на маленьких размерах — нули и время вплотную к прямому перебору (ratio 1.02 на тысяче элементов — интерфейс стал практически бесплатным).

Теперь главное — миллион элементов. Обрыв... не случился. Прочерк в Allocated, ratio 1.01. Я ждал возврата 40 байтов, перечитал пост Тауба, перепроверил цифры — обрыва

нет. Признаюсь, такого я не ожидал. Первая мысль была: может, дело в конкретной сборке рантайма? Проверяем на всех машинах, List_AslEnumerable на миллионе:

Runtime	№1 Ryzen	№2 i9	№3 Xeon	№4 Xeon
.NET 8.0	1.100 ms / 40 B	1.334 ms / 40 B	2.537 ms / 40 B	2.600 ms / 40 B
.NET 9.0	1.244 ms / 40 B	1.390 ms / 40 B	2.564 ms / 40 B	2.688 ms / 40 B
.NET 10.0	450.6 us / 0	419.7 us / 0	749.2 us / 0	749.0 us / 0

Рис. 10. List_AslEnumerable на 1 000 000 элементов, четыре машины

Нет. Три разные сервисные сборки .NET 10 — 10.0.1, 10.0.5 и 10.0.9 — и на всех нули даже на миллионе элементов. Версия рантайма как подозреваемый отпадает.

Разгадка, как я ее понимаю, в условиях замера:

- BenchmarkDotNet перед измерением вызывает метод много раз. Метод успевает полностью перекомпилироваться в tier 1 с собранным профилем — и вся цепочка девиртуализаций отрабатывает независимо от размера списка.
- Эффекты из поста Тауба бьют по другому сценарию: OSR-компиляция включается, когда метод вызвали один-два раза и внутри крутится огромный цикл — рантайм перекомпилирует его прямо посреди выполнения, и у этой OSR-версии профиля нет. В проде это типичный случай: разовая обработка большого файла, миграция, батч при старте сервиса. Бенчмарк такой сценарий физически не меряет — он всегда про горячий, много раз вызванный код.
- Пост Тауба писался по превью-сборкам, а релизные эвристики явно дотянули: три разные сервисные версии .NET 10 на четырех машинах дали одинаковые нули. Значит, дело не в конкретной сборке, а в самом сценарии замера.

Мораль третьей истории получилась не та, которую я планировал, но она даже лучше: граница у оптимизации есть и задокументирована первоисточником, но проходит она не по размеру коллекции и не по версии рантайма — а по тому, горячий у вас код или разовый.

Мой бенчмарк на четырех машинах ее не поймал, и это тоже результат: не переносите цифры прогретого бенчмарка на код, который вызывается один раз. Если у вас обрыв воспроизведется — напишите в комментариях, с какой версией и на каком размере: будет полезно всем.

Выводы

- `foreach` по конкретному типу (`List<T>`, массив, `Dictionary`) — бесплатный. `foreach` по переменной типа `IEnumerable<T>` — аллокация эnumератора, на .net 8/9 всегда, на .net 10 — как повезет с профилем и размером.
- Если пишете свою коллекцию — из публичного `GetEnumerator` возвращайте конкретный `struct`, интерфейсные версии делайте явной реализацией. Вернете интерфейс — получите свой `SortedList` с аллокацией на каждый `foreach`, и исправить это потом будет нельзя.
- `SortedList`, `ReadOnlyDictionary` и часть других коллекций BCL аллоцируют на каждый `foreach` на .net 8/9. Если такой `foreach` у вас на горячем пути — обходите по индексу или берите другую коллекцию.
- На .NET 10 не спешите переписывать код ради боксинга эnumераторов — сначала померяйте: возможно, JIT уже все убрал. Но помните, что бенчмарк меряет прогретый код: у разово вызванного метода с огромным циклом (OSR-путь) оптимизация по данным команды .NET может срываться, хотя в моих прогонах на 10.0.9 обрыв не воспроизвелся даже на миллионе элементов.
- Колонка `Allocated` в `MemoryDiagnoser` — первое, на что стоит смотреть. Ноль против ненуля важнее любых наносекунд.

Код из статьи

- `HiddenEnumerators` — бенчмарки: три класса, мульти-таргет на net8.0/net9.0/net10.0, сводка и графики после прогона
- `DisasmProof` — снятие машинного кода: программа, `snar.bat` и полные листинги `disasm_net8.txt` / `disasm_net10.txt`, из которых взяты фрагменты в статье

Ссылки

- Issue №81128 - `SortedList Enumerator` is allocated on the heap
- Issue №108913 - JIT: De-abstraction in .NET 10 (план деабстракции, синглтон пустого эnumератора)
- PR №108153 - девиртуализация интерфейсных методов массивов
- Issue №104936 - Stack Allocation Enhancements (трекинг escape analysis)
- What's new in .NET 10 runtime - официальная документация
- Stephen Toub - Performance Improvements in .NET 10 (разделы про deabstraction, OSR и PGO)
- Protobuf issue №13503 - та же дилемма со сменой сигнатуры `GetEnumerator`
- Issue №4505, открыт в 2015 — этой проблеме официально больше десяти лет

- Комментарии к статье про Dictionary и SortedDictionary на Хабре - те самые замеченные, но не объясненные 48 байтов у SortedList

Всем удачи и до новых встреч!

PS: все таблицы в статье — реальные прогоны на четырех машинах из таблицы в начале. На вашем железе абсолютные цифры будут свои — важны не наносекунды, а нули против ненулей в Allocated. Эвристики JIT меняются между сервисными релизами, так что если у вас картина другая (особенно если поймаете обрыв из третьей истории или найдете причину регрессии .net 9 из первой) — пишите в комментариях с версией рантайма, это самое интересное.

Только зарегистрированные пользователи могут участвовать в опросе. Войдите, пожалуйста.

На какой версии .NET крутится ваш прод?

0% .NET Framework (да, всё ещё)	0
0% .NET 6/7	0
0% .NET 8	0
0% .NET 9	0
0% .NET 10	0
0% Несколько версий одновременно	0

Никто еще не голосовал. Воздержавшихся нет.

Теги: benchmarkdotnet, foreach, boxing, аллокации, GC, JIT, escape analysis, .NET 10, производительность, SortedList

Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Подписаться

Оставляя почту, я принимаю Политику конфиденциальности и даю согласие на получение рассылок



5

Карма

@Geronom

Пользователь

Подписаться



 Комментировать

Публикации

ЛУЧШИЕ ЗА СУТКИ ПОХОЖИЕ



ndokutovich

10 часов назад

Теория относительности — это синус. Мнимого угла



Сложный



12 мин



9.9K



+36



37



11



Erwinmal

8 часов назад

Онигири: путь рисового колобка от японской древности до наших супермаркетов



Простой



14 мин



7K

Ретроспектива



+18



10



1



inkedsymon

9 часов назад

Мамба: архитектура, которая шла убивать трансформеры



Средний



7 мин



7.6K

Обзор



+18



14



2



monobogdan

3 часа назад

Полный разбор схемотехники кнопочного телефона. Samsung C100 — маленькое чудо корейской инженерной мысли

 Простой  11 мин  4.2K

Ретроспектива

 +16

 4

 10



kotafey

13 часов назад

LLM говнокодит не хуже людей. Только быстрее

 14 мин  9.8K

Кейс

 +16

 43

 30



BiktorSergeev

5 часов назад

Worms: как и почему игра про боевых червей уже почти 30 лет остается культовой и популярной

 8 мин  6.1K

 +10

 3

 8



IvanKlut

6 часов назад

Новая версия языка разметки текстов Markvan

 Средний  4 мин  6.5K

Кейс

 +9

 10

 8



runaway_llm

21 час назад

Энтузиаст запустил GLM-5.2 на ноутбуке с 25 ГБ RAM: без дистилляции, но на скорости от 0,05 токена в секунду

 3 мин  16K

 +9

 36

 11



Lunathecacat

4 часа назад

Лёгкий кастомный стратокастер из дешёвых комплектующих

 Простой  8 мин  4.8K

Тutorial

 +8

 2

 1



VASExperts
10 часов назад

Откуда берут начало современные QoS-решения? История вопроса — первые телефонные линии в США, эрланг и ARPANET

 Простой  6 мин  6.7K

Ретроспектива

 +7

 5

 0

Меряем глубину проникновения ИИ в разработку

Турбо

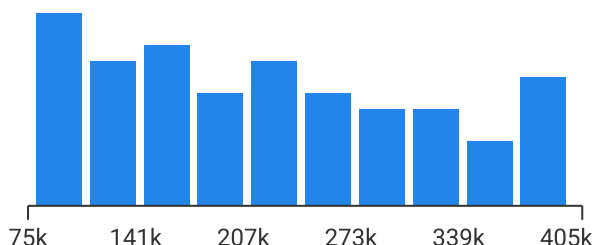
Показать еще

СРЕДНЯЯ ЗАРПЛАТА В IT

216 340

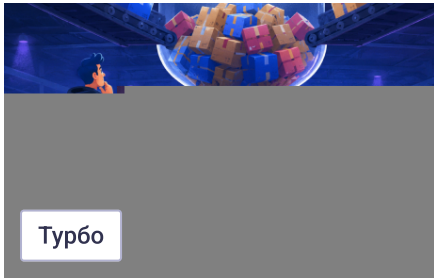
₽/мес.

— средняя зарплата во всех IT-специализациях по данным из 3 774 анкет, за 2-ое пол. 2026 года. Проверьте «в рынке» ли ваша зарплата или нет!



Проверить свою зарплату

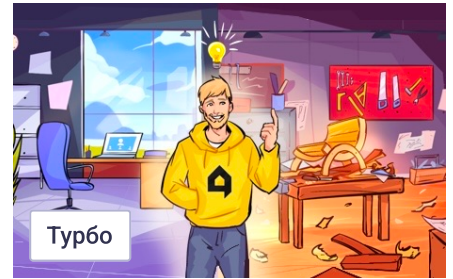
МИНУТОЧКУ ВНИМАНИЯ



Моделируем и роботизируем на хакатоне от Ozon Tech



Насколько глубоко ИИ проник в разработку? Опрос



Просто добавь IT: как совмещать код и свёрла в сезоне DIY

БЛИЖАЙШИЕ СОБЫТИЯ



2 марта – 10 августа

PROBA Awards — международная коммуникационная премия, учрежденная в 2000 году

Санкт-Петербург

Маркетинг

Больше событий в календаре



11 июня – 2 сентября

Робозон: хакатон с призовым фондом 15 млн руб.

Москва • Онлайн

Разработка

Больше событий в календаре



15 июл

Веби эпох

Онлайн

Марк

Больше



 [Настройка языка](#)

[Техническая поддержка](#)

© 2006–2026, Habr